Inspector–Executor vs. Online Scheduling for Irregular Batched Attention

Author: Kevin Ngo (kevinngo)

URL: https://kevinngonyc.github.io/projects/attention-scheduling/index.html

Summary

We are going to develop a precomputed inspector-executor and online scheduling method for irregular batched attention on GHC NVIDIA GPUs. We will compare the approaches against a naive static scheduling baseline. Performance will be analyzed primarily on attention execution time, scheduling overhead, and speedup against the baseline.

Background

$$Attention(Q, K, V) = softmax\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Attention is the primary mechanism in transformers, which is dominated by matrix multiplications. These matrix multiplications can be executed in parallel, but variable length sequences cause issues due to standard batched attention requiring uniform dimensions. The naive approach is to pad variable length sequences to the longest sequence length. This introduces inefficiencies, as much of the memory and compute can be wasted on this padding.

FlashAttention is an optimization of attention, which uses tiling to avoid storing the entire attention-score matrix in GPU memory. It deals with variable length sequences by storing them contiguously, marking the delineation between inputs and restricting attention within each one. Solutions such as these deal with variable length attention, but problems of load imbalance remain. Tiles from longer length sequences will take longer to calculate than tiles from shorter sequences, potentially leaving some SMs idle.

An inspector-executor plan precomputes a workload-aware scheduling of attention tiles to improve GPU utilization. We want to investigate online methods where tiles are assigned at runtime, to see if we can minimize the scheduling overhead of an inspector-executor approach while maintaining similar attention execution time.

The Challenge

The primary challenge is how to distribute attention workload to minimize load imbalance across SMs, caused by tiles of sequences of varying length. Another challenge is how to best tile to maximize GPU block efficiency, which will be encountered during implementation, but is not the primary learning goal. One difficulty of the load imbalance problem is that the scheduling overhead of a precomputed plan may be too costly. At the same time, coordination and the lack of global awareness of online methods can lead to more GPU underutilization. The goal is to determine when a precomputed schedule is favorable over an online schedule and how to best construct those schedules.

Dependencies. Between tiles, there are no dependencies because each query tile only reads key/value tiles, computing separate portions of the output. Within each tile computation, there are dependencies between iterations over key/value tiles in a sequence, as online softmax requires computational results from previous iterations. There is also a dependency on schedule creation for the precomputed inspector-executor approach. For the online approach, there may be dependencies with shared variables.

Memory Access Characteristics. Each thread block loads a query tile into on-chip memory once and reuses it across all key/value tile iterations. Key/value tiles have spatial locality because they are stored contiguously in memory. Many query tiles access the same key/value tiles, so these key/value tiles can be stored in cache. If query tiles from the same sequence run far apart in time, it may lead to increased L2 cache misses. Inspector–executor accesses a task schedule from memory. Online scheduling may use shared variables, introducing additional memory accesses and synchronization overhead. There is generally a low communication to computation ratio, since there is memory reuse with query tiles and caching of key/value tiles. The attention calculation tends to be compute-bound. When tiles are small or sequences are short, however, the calculation becomes cheaper and scheduling overhead remains approximately constant, leading to an increase in communication to computation ratio.

Divergent Execution. There is little warp divergence, since every thread in a block works on the same sequence. There may be some warp divergence for partial tiles. The more significant concern comes from the varying execution times of different blocks, leading to SM underutilization.

Constraints. One constraint is the amount of resources on an SM. The shared memory, registers, number of threads per block, and number of blocks that can fit on an SM affect the granularity of work distribution. There is also no control over the hardware scheduling of blocks to SMs. A schedule has to map tiles to blocks without assuming any mapping of blocks to hardware. Each task also has to run to completion and the workload cannot be balanced after it starts. This means the computation of tiles from long sequences potentially forces other SMs to remain idle. The separation of host and device memory increases latency and introduces synchronization for a precomputed inspector-executor schedule that is created on the CPU. Communication bandwidth and latency across different SMs also play a factor for any global variables related to online scheduling.

Resources

We will be using NVIDIA GeForce RTX 2080s on the GHC machines. We will implement a simplified forward-only tiled attention kernel with variable-length sequence support, using FlashAttention and FlashAttention-2 as references. FlashInfer will be used as a reference to implement inspector-executor scheduling. The implementations will use CUDA C++. Profiling will be done with NVIDIA Nsight Compute and Nsight Systems. PyTorch will be used as a correctness reference. We do not anticipate needing any additional resources at this time.

References

1. T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, vol. 35, 2022. arXiv:2205.14135.
2. T. Dao, “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning,” in *ICLR*, 2024. arXiv:2307.08691.
3. T. Zadouri, M. Hoehnerbach, J. Shah, T. Liu, V. Thakkar, and T. Dao, “FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling,” arXiv:2603.05451, 2026.
4. Z. Ye, L. Chen, R. Lai, W. Lin, Y. Zhang, S. Wang, T. Chen, B. Kasikci, V. Grover, A. Krishnamurthy, and L. Ceze, “FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving,” in *Proc. MLSys*, 2025. arXiv:2501.01005.

Goals and Deliverables

Plan to achieve.

- Simplified forward-only tiled attention kernel with variable-length sequence support, verified with PyTorch
    - Goal: within ~1e-3 max absolute error
    - Justification: around the magnitude of FP16 error
- Naive baseline: maps one query tile per block
- Inspector-executor scheduler: precompute a longest processing time first (LPT) schedule on CPU
- Online scheduler: a persistent kernel that dynamically assigns tiles, operating with global variables
- Both schedulers
    - Goal: 5% lower kernel latency than the naive baseline
    - Justification: FlashAttention-4 LPT order gives 4-8% FLOPS gain

Hope to achieve.

- If fast: split and parallelize the key/value tiles for any given query tile
- If slow: implement only the baseline and one scheduler

Analysis

- **Variables**: sequence-length distribution, sequence-length skew, batch size, schedule reuse count, tile size
- **Measurements**: kernel latency, end-to-end latency, scheduling overhead time, speedup against baseline
- When does online scheduling beat precomputed plans?
- How much inefficiency of each policy is due to imbalance vs. overhead?
- How close does each policy get to ideal?
- Does the schedule affect the L2 hit rate?
- When are scheduling costs amortized?

Platform Choice

We are using GPUs because attention is a highly parallelizable operation that maps well onto GPUs. The scheduling problem has unique GPU constraints, such as limited SM resources or hardware block scheduling, which differ from CPUs. CUDA C++ gives us direct control of the work assignment. We use the GHC machines in particular because they are easily available resources, and the general applicability of the scheduling problem does not depend on the type of GPU.

Schedule

Nov 9 - Project proposal.
Nov 16 - Simplified forward-only tiled attention kernel with variable-length sequence support, verified with PyTorch. Workload generator, lower-bound calculator, timing instrumentation.
If the kernel is not working, use synthetic tiles and potentially come back later.
Nov 23 (Project Milestone) - Naive baseline and inspector-executor scheduler. Scheduling overhead measurements, initial comparisons.
Nov 30 (Thanksgiving Week) - Basic online scheduler.
If an online scheduler is not completed, assess whether we should pivot and focus on baseline vs. inspector-executor.
Dec 7 - Final online scheduler and analysis against baseline/inspector-executor. Sweeps over skew, batch size, schedule reuse count. Nsight profiling.
Dec 10 (Final Project Report) - Perform final analyses. Compose everything into report.
Dec 11 (Poster Session)